

Implementation Details: WARG Minigames and Support Systems

Luc and Friends
Software Development Project

August 2026

1 Introduction

This document outlines the technical implementation details for the Wits Alternate Reality Game (WARG) Platform's minigames and their underlying support infrastructure.

2 Core Technology Stack

- **Backend:** Express middleware backend to handle routing and API requests.
- **Database:** MySQL to manage relational data, including creators, geographic points of interest, and real-time player states.
- **ORM layer:** Sequelize ORM to sit between the backend and database for safe data querying.
- **Frontend Geolocation:** Leaflet map interfacing directly with the device's native geolocation API.

3 Support and Auxiliary Systems

3.1 Geospatial Validation and Geofencing

The platform's core gameplay loops rely on verifiable location data.

3.1.1 Waypoint storage

MySQL contains spatial extensions which allow us to store ARG waypoints as a "POINT" geometry type. This type will utilise SRID 4326 (WGS 84).

3.1.2 Geofencing

When a user interacts with a waypoint, their device reports their GPS coordinates to the Express API. This is then queried against the waypoint's location in the database to determine whether it falls within a specified radius. MySQL provides functions to facilitate this calculation (`ST.Distance.Sphere`).

3.1.3 Frontend integration

Tracking and rendering player and waypoint locations will be handled using Leaflet maps. More specifically, by utilising the HTML5 `navigator.geolocation` API and the `watchPosition()` function to send queries to our Express endpoint.

3.1.4 Anti-Spoofing and Trust Profiles

In order to combat spoofing, we will employ various detection methods and compose trust profiles for our users. Some preliminary ideas are found below.

Drift detection: A lack of drift is an immediate flag, as spoofed locations tend not to have any drift and will report the exact same spoofed coordinate. Unspoofed locations will generally have some degree of drift as a result of noise and player movements.

Pedometer integration: By requesting access to a player's pedometer, we can detect basic spoofing technology which fails to spoof accelerometer readings. Thus, allowing us to flag spoofed journeys by detecting a lack of steps.

Speed detection: By performing a basic distance/time calculation, we can determine whether a player's speed exceeds realistic speed expectations. We can use this to raise the suspicion score on users.

3.2 Offline Resilience

To account for network dead-zones on campus, the client application will cache active puzzles (if the puzzle can be done offline). This caching will be implemented using Service Workers to intercept network requests and IndexedDB to persist puzzle states. If a player loses signal, their puzzle attempts will be held locally on the device and validated by the server as soon as the connection is restored, given the puzzle type is compatible with offline-play (live or co-op games will not be available offline).

3.3 Social and Rating Subsystem

To moderate and evaluate the minigames, the platform introduces a rating subsystem for flagging content. This is supported by an interactive commenting system allowing players to discuss, and provide feedback on the specific games they are playing. Additionally, a rudimentary friends-system will allow players to participate in co-op ARGs and track each other's progress.

4 Minigame Implementations

4.1 Scannable Asset Tags (Barcode Integration)

Creators will be able to link barcodes to puzzles within their creator studio. This scanning is facilitated using `html5-qrcode`, which will capture the barcode data and trigger a callback with the decoded data. Similarly, players will scan barcodes which, once captured, will trigger a callback to validate the decoded barcode against the data set up by the author.

4.2 AR and Environmental Mechanics (Shape Matching)

Augmented Reality (AR) modules are integrated to enable environment-driven puzzles and story-telling.

4.2.1 AR overlay viewport

Overlaying digital elements onto a live camera feed can be achieved using the HTML5 `canvas` object for the basic 2D minigames which involve overlaying silhouettes and cut-outs. This overlay viewport will be utilised in various puzzles.

4.2.2 Shape matching Engine

Depending on the compute available to our backend, we will implement a shape-matching engine using either a server-side Segment Anything Model (SAM) or a lighter-weight WebAssembly-based solution. Since Express is single-threaded, we will not be able to run SAM directly on the Node.js backend. Instead, we will utilise a lightweight Python API (e.g., FastAPI or Flask) to act as an inference microservice. Shape matching will be implemented on a fuzzy basis using the Jaccard index. If the resulting value is within a predefined threshold, the puzzle will be marked as solved.

4.2.3 Colour-Matching engine

In order to implement a colour-matching game, we will rely on an HSV colour-space encoding. By producing a colour histogram, images will produce a signature against which user-uploads can be tested. Uploads will be tested using the Bhattacharyya distance. If the Bhattacharyya distance is above a particular threshold, the puzzle will be marked as solved.

4.2.4 Basic Canvas games

Simple minigames will be produced using the HTML5 `canvas` API.

4.3 Texture Matching

Similar in spirit to colour matching, this minigame asks a player to photograph a surface (brick, concrete, cladding, cobblestone, etc.) that corresponds to a reference material set by the creator.

4.3.1 Descriptor extraction

Because raw colour histograms are insufficient to distinguish surfaces of similar hue but different material, texture matching will extract a Local Binary Pattern (LBP) histogram from the greyscale-converted upload as a lightweight, rotation-invariant descriptor of surface texture. This computation is inexpensive enough to be run either client-side (via a WebAssembly build of the LBP routine) or on the existing Python inference microservice, keeping infrastructure consistent with the shape-matching pipeline.

4.3.2 Matching

The extracted descriptor is compared against the creator-supplied reference descriptor using a chi-squared distance metric, which is standard for histogram comparisons of this kind. As with colour matching, a predefined threshold determines whether the puzzle is marked solved. Where compute budget allows, this may later be upgraded to embedding-based similarity (e.g., cosine distance between MobileNet feature vectors produced by the same microservice) without changing the client-facing API contract.

4.4 Shadow Tracing

This minigame requires the player to be physically present at a waypoint at a specific time of day and to trace the shadow currently being cast by a building or structure.

4.4.1 Expected shadow computation

Each shadow-tracing waypoint stores a building footprint polygon and an approximate height (metres) in MySQL alongside its POINT geometry. At request time, the client computes solar azimuth and elevation for the waypoint’s coordinates and the current device timestamp using a client-side solar-position library (e.g., SunCalc.js). This is combined with the stored footprint and height to project an expected shadow polygon onto the ground plane.

4.4.2 Capture and validation

The player traces the shadow they observe directly on the canvas overlay described in Section 4.2.1. The traced polygon is submitted to the Express API together with the device timestamp and geofenced location (validated as per Section 3.1.2). Because this is fundamentally a polygon-overlap problem, it reuses the existing shape-matching engine’s Jaccard index comparison rather than requiring new matching infrastructure, comparing the player’s traced polygon against the server-computed expected shadow polygon.

4.4.3 Time-gating

As the correct answer changes throughout the day and across seasons, this puzzle type is explicitly excluded from the offline caching described in Section 3.2, since a cached “expected shadow” would go stale within minutes.

4.5 Plaque and Inscription Hunt

Creators may attach trivia questions to a waypoint located at a statue, plaque, or dedication stone, encouraging players to read and engage with campus history.

4.5.1 Data model

Each waypoint of this type stores one or more question/answer pairs (short-text or multiple-choice) in MySQL, associated via a foreign key. Multiple questions per waypoint allow randomised or sequential presentation to reduce answer-sharing between players.

4.5.2 Validation

On arrival within the waypoint’s geofence, the client requests the question from the Express API; answers are validated server-side (never client-side) to prevent trivial inspection of correct answers via browser dev tools. Free-text answers are normalised (case-folding, whitespace trimming, and a small edit-distance tolerance) before comparison to tolerate minor typos.

4.5.3 Offline support

Since this puzzle type requires no live computation, question payloads are cacheable under the Service Worker/IndexedDB scheme in Section 3.2, with answer validation queued for submission once connectivity is restored.

4.6 Sign Hunt

A scavenger-hunt variant that reuses the shape-matching and colour-matching engines from Section 4.2, applied to campus wayfinding signage rather than logos.

4.6.1 Checklist structure

A sign hunt is modelled as a creator-defined checklist: a set of waypoints, each tagged with a reference image of the sign to be found. This checklist is stored as a joining table between the hunt entity and the waypoints table, allowing progress (found/not found) to be tracked per player per waypoint.

4.6.2 Matching

Player uploads are matched against the reference sign image using the same fuzzy shape-matching (Jaccard index) and, where the signage system uses consistent brand colours, the colour-matching pipeline as a secondary check. A sign is marked “found” once both checks clear their respective thresholds, and the hunt as a whole completes once all checklist items are found.

4.7 Shortest Path Finder

Players are asked to identify or walk the shortest real-world route between two campus landmarks.

4.7.1 Canonical path storage

The walkable path network between landmarks is pre-computed by creators (or derived from OpenStreetMap campus path data) and stored as a MySQL LINESTRING geometry per route, alongside the two endpoint waypoints.

4.7.2 Two capture modes

Drawn mode: the player sketches their proposed route directly on the Leaflet map without walking it, submitting a client-side polyline. **Walked mode:** the player’s route is recorded as a breadcrumb trail via `watchPosition()` (Section 3.1.3) as they physically walk between the two landmarks.

4.7.3 Validation

In both modes, the submitted path is compared against the canonical LINESTRING using a discrete Fréchet distance calculation (via Turf.js on the client for immediate feedback, with a server-side recomputation for authoritative validation). A path within a tolerance threshold of the canonical route, and whose total length is within a small margin of the shortest-path length, is marked correct. Walked-mode submissions are additionally subject to the speed and drift anti-spoofing checks in Section 3.1.4.

4.8 Height Guesser

Players estimate the height of a building, statue, or landmark using an in-AR measuring tool.

4.8.1 AR measurement

This relies on the device’s AR hit-testing capability (WebXR Device API where available) to place two anchor points in real-world space: one at the base and one at the top of the target structure, using the phone’s tracked camera pose to triangulate the real-world distance between them.

4.8.2 Fallback for unsupported devices

On devices without WebXR hit-testing support, a simpler reference-object method is used: the player places a virtual object of known height (sourced from the creator’s data) next to the structure in the canvas overlay and adjusts a scale slider until the object visually matches a known reference in frame, from which height is inferred proportionally.

4.8.3 Validation

The player’s estimate is compared against the true height stored against the waypoint in MySQL, with the puzzle marked solved if the guess falls within a creator-configurable percentage tolerance (e.g., $\pm 10\%$).

4.9 Then-and-Now AR Overlay

Creators upload an archival photograph of a location taken from a fixed vantage point; players must stand at that vantage point and align their live camera feed with the historical image.

4.9.1 Vantage point and pose data

Each archival photo waypoint stores the geofenced standing location, a target compass heading, and the archival image itself. The compass heading is compared against the device orientation API to guide the player toward the correct facing direction before the overlay is shown.

4.9.2 Alignment overlay

Once within the geofence and facing approximately the correct heading, the archival image is rendered as a semi-transparent layer on the canvas overlay (Section 4.2.1), allowing the player to physically move until the historical structure lines up with its present-day counterpart.

4.9.3 Capture validation

On capture, the live frame is sent to the same Python inference microservice used for shape matching, where ORB or SIFT keypoint matching is performed between the live frame and the archival photo. A minimum count of well-matched, geometrically consistent keypoints is used as the success threshold, since raw pixel similarity is unreliable across a photo separated by years or decades and differing lighting conditions.

4.10 Symmetry Finder

Players identify an axis of symmetry in a symmetrical architectural feature (arches, courtyards, facades) and trace it on the live camera feed.

4.10.1 Axis capture

The player draws a single line on the canvas overlay indicating the perceived axis of symmetry over a captured frame.

4.10.2 Validation

The captured frame is mirrored about the drawn axis, and the mirrored half is compared against the opposite half of the original image using a structural similarity index (SSIM). This computation is delegated to the Python inference microservice alongside the shape- and texture-matching workloads, keeping all image-comparison logic centralised in one service. A high SSIM score between the mirrored and actual halves marks the puzzle solved, with a threshold that is deliberately generous to account for minor camera skew and imperfect real-world symmetry.

4.11 PvP and Social Minigames

The following minigames build directly on the Synchronisation infrastructure described in Section 5, and additionally make use of the Social and Rating Subsystem (Section 3.3) for team formation and content moderation.

4.11.1 Point Domination

Contested campus zones (courtyards, quads, plazas) are modelled as MySQL POLYGON geometries rather than single waypoints, each associated with a controlling faction or team.

Capture logic: while a team's players are physically present within a zone's geofence, a capture-progress value accrues on a per-tick basis. Ties and contested captures (players from opposing teams present simultaneously) pause or reverse progress rather than favouring either side, to discourage camping.

Live-play integration: this follows the tick-based live-play model in Section 5.2, using Socket.io to broadcast updated zone-ownership state to all players within range at the end of each tick, with clients interpolating between broadcast states for smooth UI updates.

Persistence: zone ownership and capture history are written to the database on ownership change (not every tick) to avoid excessive write load, consistent with the persistence approach used for turn-based games in Section 5.1.

4.11.2 Landmark Relay

A team-based hunt in which each team member is assigned a different waypoint; visiting it and passing its geofence check yields a fragment (partial code, image tile, or puzzle piece) rather than solving the puzzle outright.

Turn/coordination model: this is implemented as a turn-based game per Section 5.1, with the Express server holding the authoritative relay state (which fragments have been collected, by whom) as a JSON object, and validating each fragment collection server-side against the reporting player’s geofence.

Assembly: once all assigned fragments for a team have been collected, the server assembles and unlocks the combined result (e.g., a final code or composite image) and broadcasts completion to all team members via WebSocket.

Offline caveat: as with other live/co-op games, Landmark Relay is excluded from the offline caching described in Section 3.2, since fragment state must remain synchronised across teammates in real time.

4.11.3 AR Photo-Bombs

A player (or a creator, as a seeded challenge) places a virtual object or mascot at a specific real-world pose — position and orientation — at a waypoint. Other players must locate it in AR and reproduce a matching “photo” of it.

Placement data: the placing player’s device pose (derived from WebXR/ARCore tracking) at the moment of placement is stored alongside the waypoint, rather than only a flat GPS point, since the challenge is about vantage point as well as location.

Discovery and validation: rather than relying purely on image similarity (as in the Then-and-Now overlay), validation primarily compares the finding player’s device pose against the stored placement pose within a tolerance, since pose data is more robust and less compute-intensive than keypoint matching at scale. The resulting in-AR screenshot is still captured and stored as user-generated content.

Moderation: captured photo-bomb images are surfaced through the Social and Rating Subsystem (Section 3.3), allowing other players to rate or flag submissions, and allowing creators to moderate seeded challenges.

4.12 Synchronisation (Co-op Puzzles)

Live-play mechanics will allow multiple players to participate in collaborative or competitive ARG instances concurrently.

4.12.1 Turn-Based games and puzzles

Servers will act as the authority on game state and rules. The Express server will represent the game-state as a JSON object and will verify player actions when a move is submitted. This will be implemented over a WebSocket protocol for concurrent games. For every validated move, data is written to a table in the database in order to persist in the event of a lost/spotty connection.

4.12.2 Live-play games and puzzles

Utilising an appropriate tick speed, player inputs will be transmitted and processed. At the end of each tick, a new game state will be broadcast to the players in the geofenced location. This will be implemented using WebSockets (such as Socket.io) and dumb clients. In order to smooth the transition between game-ticks, rendering will occur at a higher refresh rate and states will be interpolated between each other.

AI Declaration: The preceding document was partially generated, reviewed and edited with: Gemini Web[Gemini 3.1 Pro]